Section 21

Shortest Paths

One of the classic examples, if not *the* classic example, of an algorithmic problem is route planning: We are provided cities, say A to Z, and distances between the cities (as shown in Figure 21.1, and we want to find out what the shortest path from city A to city Z is.

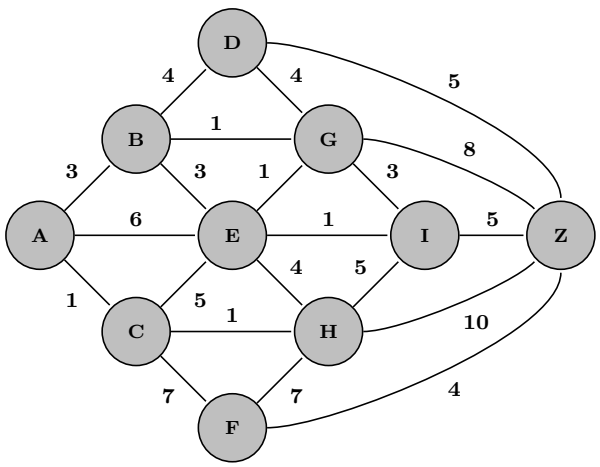


Figure 21.1: Cities A to Z and their distances. What is the shortest path from A to Z?

Subsection 21.1

Weighted Graphs and Shortest Paths

To study this problem, we have to introduce the notions of **weights** and **weighted graphs**. A weighted graph $G = (V, E, c)$ is a graph $G = (V, E)$ with a weight function $c\colon E \to \mathbb{R}$ that assigns each edge a weight. We call $c(e)$ the weight (sometimes also cost, hence the letter) of the edge e. See Figure 21.2 for an example of a weighted graph.

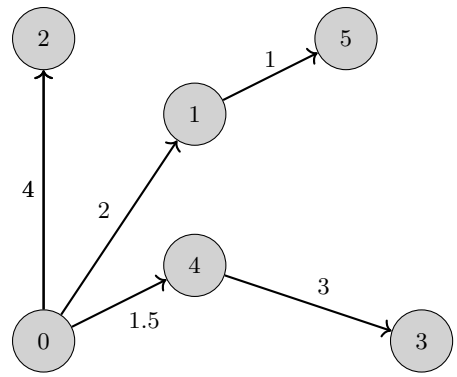


Figure 21.2: A weighted graph: We indicate the weight of an edge by writing the number somewhere in the edge's vicinity. For instance, we have $c((0,4)) = 1.5$.

Given a weighted graph, we can now study the **weight of a path**. Let us consider a path p from s to t, thus a sequence $(v_1, \ldots, v_{k+1})$ for some $k \geq 1$ with $(v_i, v_{i+1}) \in E$, as well as $v_1 = s$ and $v_{k+1} = t$. The weight (sometimes also referred to as cost of the path) of p is defined as the sum of the weights of its edges, i.e.,

$$c(p) := \sum_{i=1}^{k} c\left((v_i, v_{i+1})\right).$$

See Figure 21.3 for an example.

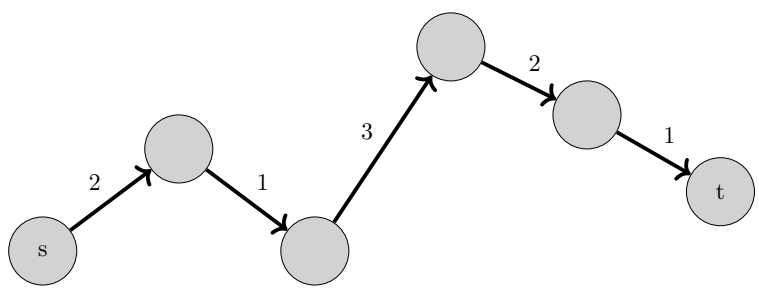


Figure 21.3: A path from s to t with weight 9.

To simplify the notation, we usually write $p\colon s \rightsquigarrow t$ or $s \overset{p}{\rightsquigarrow} t$ to indicate that p is a path from s to t, especially when the intermediate nodes do not matter that much.

The route planning problem can now be formulated as finding the path with minimum weight from a start node s to a target node t. Slightly abusing notation, this path is usually referred to as **shortest path** and its weight is called its **length**. The shortest path in a weighted graph G from node s to node t thus is defined as the path p with minimum weight $\delta(s,t)$, where

$$\delta(s,t) = \begin{cases} \infty & \text{no path from } s \text{ to } t \\ \min\{c(p)\colon s \overset{p}{\rightsquigarrow} t\} & \text{otherwise} \end{cases}$$

Subsection 21.2

Simple Observations

Before looking at algorithms, let us first make some simple observations. One question that one might ask: Is the shortest path always unique? The answer is no. There might be multiple paths with the same weight. An example is depicted in Figure 21.4. Our goal will be to just find one shortest path, no matter which.

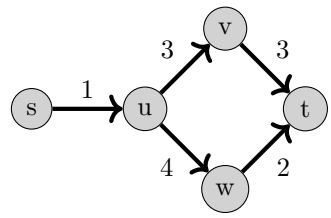


Figure 21.4: A graph with two shortest paths from s to t: (s, u, v, t) and (s, u, w, t) both have length 7.

Another obvious question is: Does a shortest path always exist? Of course, if there are only finitely many paths, then there must exist a shortest one. But what if there are infinitely many? How would that even be possible in a finite graph? If the graph has a cycle, then indeed there might be infinitely many paths from s to t by going through the cycle arbitrarily many times. Why should we want to do that when looking for the shortest path? If the total weight of the cycle is positive, then obviously we would not want to do that. But what if the weight of the cycle is negative (we call such a cycle a **negative cycle**)? Each time we go through the negative cycle, our path gets shorter and shorter. There is no shortest path! An example can be found in Figure 21.5. Note that for the application of route planning, negative weights do not make much sense, but there are other real-world settings where negative weights occur.

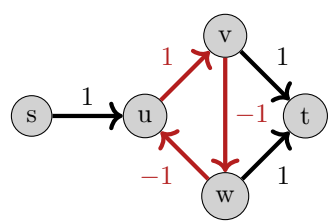


Figure 21.5: Since there is a negative cycle (u, v, w, u) with weight -1. There is no shortest path between s and t. Not going through the cycle at all leads to a weight 3, going through the cycle once yields weight 2, using the cycle twice results in weight 1, and so on.

We observe the following.

Observation 21.1. *A shortest path does not contain any cycles. Indeed, a cycle either has negative weight, in which case no shortest path exists; positive weight, in which case omitting the cycle would make the path shorter, leading to a contradiction; or weight* 0*, in which case we can omit the cycle (by convention) without changing the weight.*

Let us suppose that a shortest path exists, that is, that our graph has no negative cycle. What could we do? There are only finitely many paths, so we could just try all of them? This brute-force algorithm is incredibly inefficient. Unfortunately, there can be many paths in a graph. Finitely many, but exponentially many. For instance, in a complete graph (with all possible edges) on n nodes, there are $(n-2)!$ many paths between any two nodes. See Figure 21.6 for a more interesting (or realistic, at least) example.

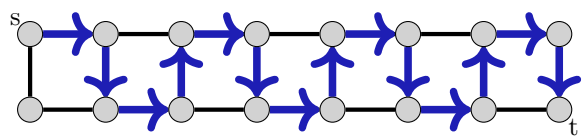


Figure 21.6: An undirected graph with exponentially many paths between s and t: a grid of size $2 \times n/2$ (in the drawn graph $n = 16$). Let us focus on simple paths only; there are even more paths with repeating nodes. We have to go right $n/2-1$ times. For each of those moves to the right, we can decide whether we want to move at the top or at the bottom. Thus, overall, we have at least $2^{n/2-1} \in \Theta\left(2^{n/2}\right)$ many such options, hence at least that many paths from s to t. One such path (that goes right alternately at the top and at the bottom) is depicted in blue.

What other possibilities, besides brute-force, do we have? Do we maybe know how to at least solve a special case of the problem? What about all edges having the same weight? Or, to make it slightly more obvious, if all edges have weight 1? We have seen that BFS can be used to find the path with the fewest number of edges. If all the weights are 1 or the same (positive!, why?) constant, this is equivalent to finding the path with the minimum weight, thus the shortest path. As a brief reminder, BFS starting from node s explores the graph by first visiting s, then visiting all out-neighbors of s, then all out-neighbors or out-neighbors of s, and so on. It thus visits the graph in layers of increasing distance (in terms of number of edges) in the graph. Once it encounters the node t, it has found a shortest path from s to t in number of hops, which corresponds to the shortest path (in terms of weight). See Figure 21.7 for a schematic visualization.

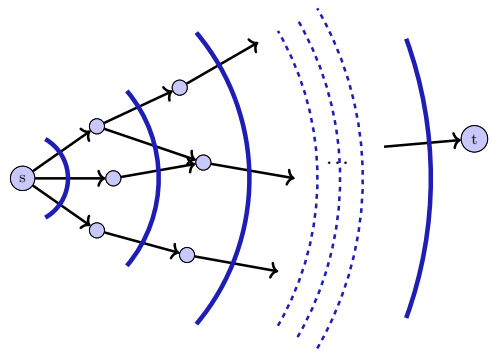


Figure 21.7: BFS explores the (unweighted) graph layer by layer in increasing distance (in terms of number of edges).

Thus, the problem on unweighted graphs can be solved in $\Theta(|V| + |E|)$ time. Can we also do that when the graph is weighted? Obviously, BFS does not work. The shortest path in terms on number of edges does not need to be the shortest path in terms of weight.

We start our quest for an efficient shortest-path algorithm in weighted graphs with the following observation: Shortest paths obey the principle of the optimal substructure (we will discuss this later in more detail when talking about Dynamic Programming).

Observation 21.2. *If the shortest path p from s to t goes via a node v, then p must contain the shortest path from s to v and the shortest path from v to t. More formally, if $p = s \overset{p_1}{\rightsquigarrow} v \overset{p_2}{\rightsquigarrow} t$, then p_1 must be the shortest path from s to v and p_2 must be the shortest path from v to t. More generally, if $p = (v_1, \ldots, v_k)$ is a shortest path, then all subpaths $p_{ij} = (v_i, \ldots, v_j)$ for $1 \leq i \leq j \leq k$ are shortest paths as well.*

Proof. Suppose towards a contradiction that the subpath p_{ij} of a shortest path p, for some i and j, is not optimal, thus that there exists a path p'_{ij} connecting v_i and v_j with $c(p'_{ij}) < c(p_{ij})$. Then $p' := v_1 \overset{p_{1i}}{\rightsquigarrow} v_i \overset{p'_{ij}}{\rightsquigarrow} v_j \overset{p_{jk}}{\rightsquigarrow} v_k$ is shorter than p, since $c(p') = c(p) - c(p_{ij}) + c(p'_{ij}) < c(p)$, contradicting the optimality of p. See Figure 21.8 for an illustration. □

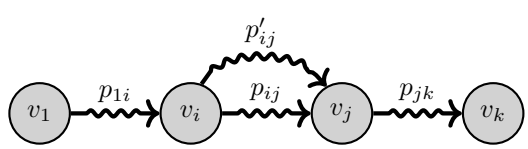


Figure 21.8: If $p = (v_1, \ldots, v_k)$ is a shortest path, then $p_{ij} = (v_i, \ldots, v_j)$ must be a shortest path between v_i and v_j. Otherwise, if there is a shorter path p'_{ij} from v_i to v_j, the path $p' := v_1 \overset{p_{1i}}{\rightsquigarrow} v_i \overset{p'_{ij}}{\rightsquigarrow} v_j \overset{p_{jk}}{\rightsquigarrow} v_k$ would be shorter than p.

Due to this observation, computing the shortest paths from a single source s to a single target t essentially requires us to compute all shortest paths from a single source to all possible targets in V. We thus now consider this more general problem of **single-source shortest paths**.

Subsection 21.3

Dijkstra's Algorithm

In the following, we make (for now) a crucial assumption: no negative weights. We will discuss the consequences of removing this assumption (hence discuss the general case of arbitrary weights) later on. Note that when studying the problem of the shortest path, we can assume without loss of generality that there are no edges of weight 0. If there was an edge (u, v) with weight 0, we could just merge u and v to a single new node w, setting $N^-(w) := N^-(u) \cup N^-(v)$ and $N^+(w) := N^+(u) \cup N^+(v)$. We thus now consider a weighted graph $G = (V, E, c)$ with positive weights. Our goal is to find all shortest paths from a single source node s to all nodes in V.

We want to devise an iterative procedure that finds the shortest path from s to a new node in every iteration. How do we start? Initially, we know the shortest path from s to s. It has length $\delta(s, s) = 0$. Amazing! What can we do next? We could look at the outgoing edges from s. Every outgoing edge (s, v) presents us with a path (of length 1) from s to that node v. The shortest path from s to v thus clearly must have length at most $c((s, v))$. See Figure 21.9.

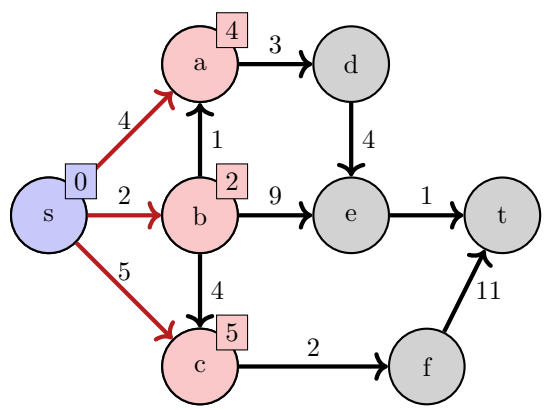


Figure 21.9: Initially, we know the shortest path from s to s (indicated by the blue color; the blue box contains the length $\delta(s, s)$ of a shortest path). By looking at the outgoing edges of s, we find upper bounds on the lengths of the shortest paths to these nodes (indicated by the red color; the red box contains an upper bound on the length $\delta(s, \cdot)$ of the shortest path from s to that node). For instance, the shortest path from s to a has length at most 4. However, there might be shorter paths. For instance, the path (s, b, a) has length 3 only.

These upper bounds are just that: upper bounds. There could be shorter paths than this. However, let us now look at the node v with the smallest upper bound. Could we reach v from s via a path with shorter length than that? No! The shortest outgoing edge is the shortest path to that node. Any path from s will next have to go to one of the out-neighbors of s, thus use one of these edges. If (s, v) is the shortest outgoing edge, there cannot be a shorter path to v than that. See Figure 21.10. Here, we crucially make use of the assumption that all weights are positive.

Let us summarize this finding in the following observation:

Observation 21.3. *Let v be the out-neighbor of s with minimum edge weight, i.e., $c((s,v)) \leq c((s,w))$ for all $w \in N^+(s)$. Then the shortest path from s to v is $c((s,v))$.*

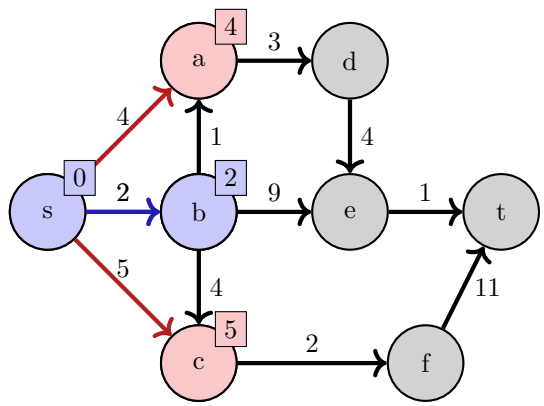


Figure 21.10: The shortest outgoing edge is the shortest path to that node. Since (s,b) has the smallest weight (2), we know that the shortest path from s to b must have length 2. Using any other edge and then somehow coming back to b would lead to a larger weight (as there are no negative weights!).

After this, we now have found the shortest path from s to another node v and we know upper bounds $d_s[x]$ on $\delta(s,x)$ for all $x \in N^+(s)$. What can we do next? We can repeat the same thing! We look at the outgoing edges of the node v and update the upper bounds of its out-neighbors accordingly. More precisely, if $d_s[v] + c((v,w))$ for $w \in N^+(v)$ is smaller than the current upper bound $d_s[w]$ of w (we set $d_s[\cdot] := \infty$ initially, meaning that we have not found an upper bound yet), we update the upper bound of w to $d_s[w] \leftarrow d_s[v] + c((v,w))$. See Figure 21.11.

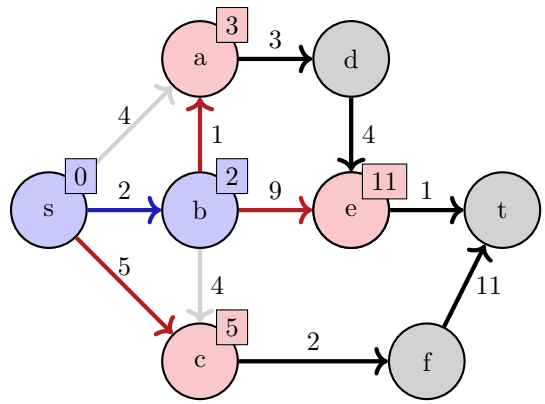


Figure 21.11: We look at the outgoing edges of b. Weight 1 to a: This improves the upper bound of a from 4 to $1 + 2 = 3$. We can update the upper bound and forget about the previous upper bound (the edge (s, a) of the previous upper bound is now marked in grey). Weight 9 to e: This gives a new upper bound of $2 + 9 = 11$ for e. Weight 4 to c: This would give an upper bound of $2 + 4 = 6$ which is worse than the current upper bound. We thus leave the upper bound unchanged and mark the edge (b, c) grey (to indicate that it is useless for us!).

We now have the shortest paths to s and v as well as upper bounds $d_s[\cdot]$ on all nodes in $(N^+(s) \cup N^+(v)) \setminus \{s, v\}$ with $d_s[\cdot] \geq \delta(s, v)$. This is because we chose v with the minimum $d_s[v]$ and later (potentially) updated the values $d_s[x]$ to $d_s[v] + c((v, x)) > d_s[v]$ for $x \in N^+(v)$. We note that $d_s[x]$ is the length of the shortest path from s to x that only potentially uses v as intermediate node, thus the shorter[32] of (s, x) and (s, v, x).

Let w be the node (different from s and v) with the smallest upper bound $d_s[w]$. We can conclude that $d_s[w] = \delta(s, w)$. Why? As mentioned before $d_s[w]$ is the length of the shortest path from s to w using either no intermediate node or using v as intermediate node. If $\delta(s, w) < d_s[w]$, there would need to exist a shortest path p from s to w using a different second node, say z, leading to the path $s \to z \rightsquigarrow w$ (a squiggly arrow means path of arbitrary length, while a straight arrow means a single edge, thus a path of length 1). But $c((s, z)) \geq d_s[z] \geq d_s[w]$, so the path $s \to z \rightsquigarrow w$ has cost $\geq c((s, z)) \geq d_s[w]$. So the path via z cannot be better! We thus have found the shortest path to yet another node!

We repeat this—i.e., picking the node with the smallest upper bound, looking at its out-neighbors and updating their upper bounds—until all nodes have been picked! Before looking at an example of this greedy (as we are greedily picking the smallest upper bound in every step) algorithm, we want to generalize the observation above (and also the one from Observation 21.3). So far, we have only argued about the correctness of the first three steps (including the trivial one of the shortest path to s), so now let us argue about the correctness of each step!

For that, we introduce the following notation to describe the set-up of the algorithm. Consider an arbitrary step of the algorithm. The nodes V are partitioned into three subsets, S (shortest path from s to this node known; these nodes already have been picked), $U := \bigcup_{v \in S} N^+(s)$ (upper bound $< \infty$ for shortest path from s to this node known), and

[32] For simplicity of notation, setting path lengths to ∞ if they do not exist.

$R := V \setminus (S \cup U)$ (remainder). We start with $S = \emptyset$, $U = \{s\}$ (we initially know an upper bound 0 for the shortest path from s to s), and $R = V \setminus \{s\}$, and will end with $S = V, U = \emptyset$, and $R = \emptyset$ (in the end, we know the shortest paths to all nodes). See Figure 21.12 for an example.

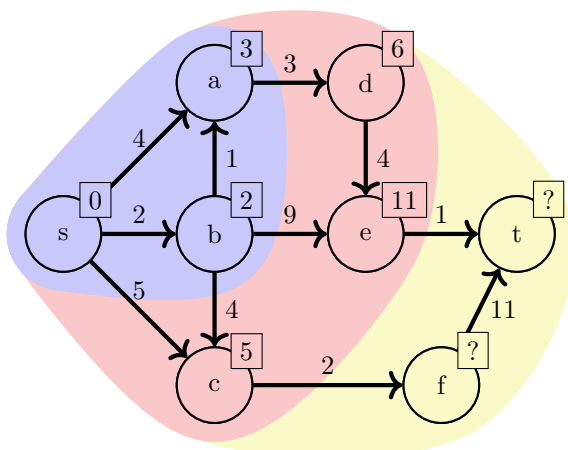


Figure 21.12: The nodes are partitioned into three sets: the set S with known shortest path (blue); the set U of nodes reachable from S (red) for which we know an upper bound; and the set $R := V \setminus (S \cup U)$ (yellow) for which we do not know an upper bound (only ∞).

The greedy algorithm from above (due to Dijkstra) can be reformulated as follows: Starting with $S = \emptyset$, $U = \{s\}$, and $R = V \setminus \{s\}$, we repeat the following until $S = V$ and hence $U = \emptyset$: In each iteration, we add the node u from U with the smallest upper bound $d_s[u]$ to S (and remove it from U), set its shortest path length to $d_s[u]$, and update d_s for all nodes in $N^+(u)$. More concretely, the update looks as follows: If there is an edge (u, a) to a node $a \in S$, then this edge is ignored; if there is an edge to a node $b \in U$ with $d_s[b] \leq d_s[u] + c((u, b))$, then this edge is ignored; if there is an edge to a node $c \in U$ with $d_s[c] > d_s[u] + c((u, c))$, then the upper bound of c is updated to $d_s[c] \leftarrow d_s[u] + c((u, c))$; and if there is an edge (u, d) to a node $d \in R$, then d is added to U, and its upper bound is set to $d_s[d] \leftarrow d_s[u] + c((u, d))$. See Figure 21.13.

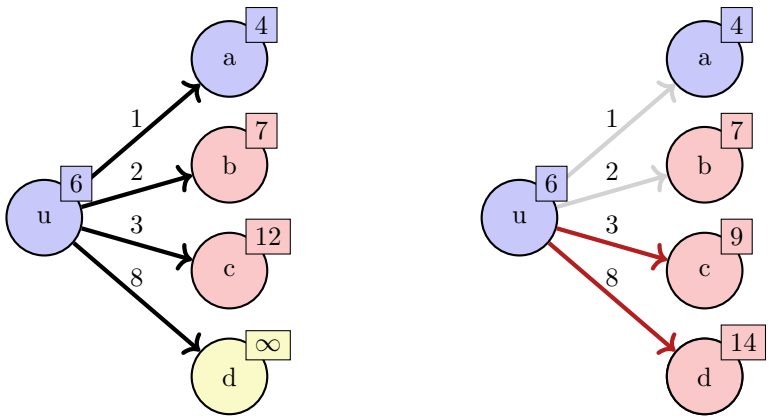


Figure 21.13: The nodes in $N^+(u)$ before and after the update of d_s when looking at the outgoing edges of u after having newly added u to S.

After i iterations, we have $|S| = i$. Hence, the algorithm terminates after n iterations and claims to have found the shortest path to all nodes. Why is this algorithm correct?

We show an invariant that the nodes are added to S in increasing order of their shortest-path-length from s. We note that it is easy to see that the nodes are added to S in increasing order of their upper bound $d_s[\cdot]$ (referred to as monotonicity of d_s). Why? The upper bound $d_s[v]$ of a node v is only updated (potentially) when a predecessor u of this node w moves from U to S. In that case, w gets an upper bound that is strictly larger than the upper bound of v.

Lemma 21.1. *Let $v_1, \ldots, v_n$ be the order in which the nodes are added to S. Then $\delta(s, v_i) \leq \delta(s, v_j)$ for $1 \leq i < j \leq n$ and $d_s[v_i] = \delta(s, v_i)$ in iteration i.*

Proof. We show the claim by induction. For $i = 1$, obviously, $v_1 = s$ is the node with the shortest distance from s and $d_s[s] = 0 = \delta(s, s)$. We now assume that $\delta(s, v_1) \leq \ldots \leq \delta(s, v_i)$ as well as $d_s[v_j] = \delta(s, v_j)$ for $1 \leq j \leq i$. We show that $d_s[v_{i+1}] = \delta(s, v_{i+1}) \geq \delta(s, v_i)$.

Consider a shortest path p from s to v_{i+1}. We show that it has length at least $\delta(s, v_i)$. Consider the first node w on p that is not in S in the beginning of iteration $i+1$ (potentially $w = v_{i+1}$). Its predecessor is in S by definition, hence a node v_k for $1 \leq k \leq i$. We thus can write $p = s \rightsquigarrow v_k \to w \rightsquigarrow v_{i+1}$. By the optimal substructure (see Observation 21.2), the subpath $s \rightsquigarrow v_k$ must be a shortest path, i.e., $c(s \rightsquigarrow v_k) = \delta(s, v_k)$. Using the induction hypothesis, we can conclude that $\delta(s, v_k) = \delta'(s, v_k)$. Moreover, since w is added to U at the latest when v_k is added to S, we have (by the update rule) that $d_s[w] \leq d_s[v_k] + c((v_k, w))$. Finally, observe that in iteration $k+1$, node v_{k+1} is added to S, hence $d_s[v_{k+1}] \leq d_s[w]$ by the greedy choice. Taken together, we thus observe that

$$\begin{aligned} c(p) &= c(s \rightsquigarrow v_k) + c((v_k, w)) + c(w \rightsquigarrow v_{i+1}) \\ &\geq c(s \rightsquigarrow v_k) + c((v_k, w)) = d_s[v_k] + c((v_k, w)) \geq d_s[w] \geq d_s[v_{k+1}], \end{aligned}$$

which implies that $d_s[v_{k+1}] = \delta(s, v_{k+1})$. The property $\delta(s, v_{k+1}) \geq \delta(s, v_k)$ follows now

from the monotonicity of d_s (and the induction hypothesis which tells us that $d_s[v_k] = \delta(s, v_k)$). $\square$

Observe that by how the upper bounds d_s for the nodes in U are computed, $d_s[u]$ for a node $u \in U$ is the length of a shortest path from s to u that uses only nodes from S. Another invariant of the algorithm!

Note that Lemma 21.1 implies that constellations as the one in Figure 21.14 are not be possible: the upper bounds as well as the actual shortest path distances are monotonically increasing over time. If a node is in S, it has to have shorter distance to s than any node not in S yet.

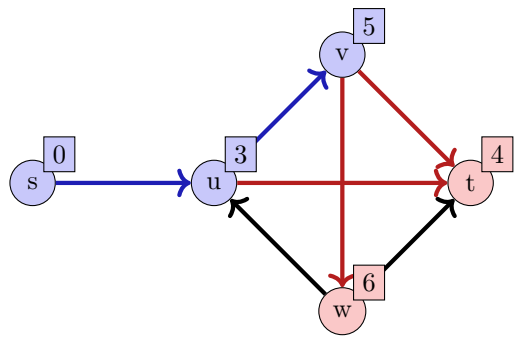


Figure 21.14: An impossible constellation during Dijkstra's algorithm: nodes are added to S in increasing order of their shortest path distance to s (monotonicity!). No node outside S can have shorter distance, otherwise it would have been added to S before. So it is impossible that t with upper bound 4 is still in U (red) while v with distance 5 has been added to S (blue).

Let us now look at an example step for step in Figure 21.16. Every node w knows the edge (v, w) that lead to the current value of $d_s[w]$. When w is added from U to S, node w knows that this node v is its predecessor on the shortest path from s to w. By remembering all predecessors $\pi_s \colon V \to V$, it is thus possible to reconstruct not only the length of the shortest path but also the actual path: The shortest path from s to t is obtained by following the predecessors, starting from t, ending at s, and reporting the encountered nodes in reverse order. This, for instance, can be easily achieved with a stack! In Algorithm 21.1, the pseudo-code of Dijkstra, including keeping track of the predecessors, can be found.

Input: Weighted Graph $G = (V, E, c)$ with $c \geq 0$, starting point $s \in V$
Output: Length d_s of the shortest paths from s and predecessor π_s for each node

```
foreach u ∈ V do
  | d_s[u] ← ∞; π_s[u] ← null
end
d_s[s] ← 0; U ← {s}
while U ≠ ∅ do
  | u ← arg min_{u∈U} d_s[u]; U ← U \ {u}
  | foreach v ∈ N⁺(u) do
  |   | if d_s[u] + c(u, v) < d_s[v] then
  |   |   | d_s[v] ← d_s[u] + c(u, v)
  |   |   | π_s[v] ← u
  |   |   | U ← U ∪ {v}
  |   | end
  | end
end
```

Algorithm 21.1: Dijkstra

We observe that we do not have to explicitly keep track of the sets S and R, it is enough to store U. When meeting an out-neighbor v of u that is in S, then the if-condition will be false (due to the monotonicity in Lemma 21.1), so no updates will happen. And due to the initial value ∞ for d_s, the if-condition will be always true for nodes $w \in R$.

But which data structure do we use for U? It should support the operations `Insert((p, k))` (add a key (node) k with value (upper bound) p) and `ExtractMin()` (remove smallest key). This corresponds to the abstract data type priority queue, which can be implemented using a min heap: The nodes in U are added to a min heap with their upper bound $d_s[\cdot]$ as priority. Note that in order to know which node this priority belongs to, we also have to store the node (i.e., its index or name) with its priority. See Figure 21.15.

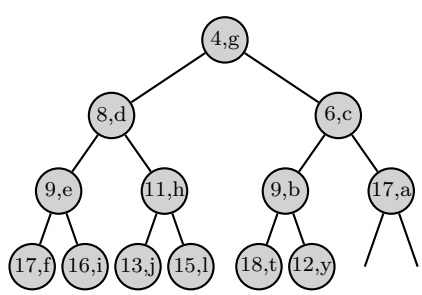


Figure 21.15: A min heap of the nodes in U with their upper bound $d_s[\cdot]$ as priorities. To know which node this priority (and hence upper bound d_s) belongs to, we also store the name (index) of the node with the key.

Insertion and extraction of the minimum can be done in $\mathcal{O}(\log |V|)$ time (as the heap consists of at most $|U| \leq |V|$ elements). But there is a caveat! Our data structure needs to support more than just insertion and extraction: we also have to update the priority of an element. More precisely, for an element (p, k) (node k with priority p) we need to decrease its priority

to p'. How would we do that in a heap? We would first need to search for the node k in the heap (in $\Theta(|V|)$ time), change the priority from p to p', and possibly adapt the structure of the heap (in $\mathcal{O}(\log |V|)$). As we potentially have many such updates (as many as $|E|$, as we will discuss later), we cannot afford linear running time per update.

There are two options how to avoid the search for a node in the heap: The first option would be to store the position of a node in the heap either in the node or in an external array. The search for a node then can be done in $\Theta(1)$ time, so the overall update in $\Theta(\log(|V|))$. We have $|V|$ insertions and extractions (every node is inserted and extracted exactly once), and potentially $|E| - |V| + 1$ decrease-key operations (every edge can lead to a decrease-key operation, except for $|V| - 1$ many that go to nodes in R, and hence lead to an insertion). This results in running time $\mathcal{O}\left((|V| + |E|)\log(|V|)\right)$, which can be written as $\mathcal{O}\left(|E|\log(|V|)\right)$ under the assumption that the graph is connected, hence $|E| \geq |V| - 1$. Disadvantages are the additional memory as well as the additional update time: Whenever a node is moved around in the heap, its position needs to be updated.

Another (more commonly seen) option is **Lazy Deletion**. Whenever we want to decrease the priority of a node k from p to $p' < p$, we leave the element (p, k) untouched and add an additional element (p', k) to the heap. When a node is extracted from the min heap for the first time (with its smallest priority), we can mark it as deleted. When we later encounter the node with a larger priority, we can ignore it and move to the next minimum. See Algorithm 21.2 for the code.

```
Input: Weighted Graph G = (V, E, c) with c ≥ 0, starting point s ∈ V
Output: Length d_s of the shortest paths from s and predecessor π_s for each node
foreach u ∈ V do
    d_s[u] ← ∞;  π_s[u] ← null
end
S = {}; d_s[s] ← 0; U ← {s}
while U ≠ ∅ do
    d, u ← ExtractMin(U)
    if u ∉ S then
        S ← S ∪ {u}
        foreach v ∈ N⁺(u) do
            if d + c((u, v)) < d_s[v] then
                d_s[v] ← d + c((u, v)); π_s[v] ← u
                Insert(U, (d + c((u, v)), v))
            end
        end
    end
end
```

Algorithm 21.2: Dijkstra with lazy deletion

Note that since, for lazy deletion, we have to keep track of the nodes that have already been extracted from the heap (which are the nodes in S), we now also explicitly store S in addition to U. We could do that using, for instance, a boolean array, with an entry at position i equal to 1 if and only if the node with that index is in S. This guarantees constant

insertion and searching time for S.

The main advantage of lazy deletion is that we can use the priority queue (the heap) as a black box and do not need to have access to its implementation (details); we do not need to keep track of the positions of the nodes. The main disadvantage is the size of the heap: It could grow from $|V|$ to $|V| + |E|$, since potentially every edge could lead to an improvement in the upper bound of a node (and hence to the insertion of a new element to the heap). Fortunately, since $|E| \leq |V|^2$, the insertion and extraction time stays logarithmic in $|V|$, observing that $\mathcal{O}(\log(|V|^2)) = \mathcal{O}(\log(|V|))$. How many insertions/extractions do we have? At most $|E| + 1$ many. We thus get a running time of $\mathcal{O}\left(|E| \log(|V|)\right)$ with extra memory $\mathcal{O}(|V| + |E|)$.

We will see that by using Fibonacci heaps—which can perform the decrease-key operation in $\mathcal{O}(1)$ amortized time—instead of regular min heaps, we can reduce the running time to $\mathcal{O}\left(|E| + |V| \log(|V|)\right)$.

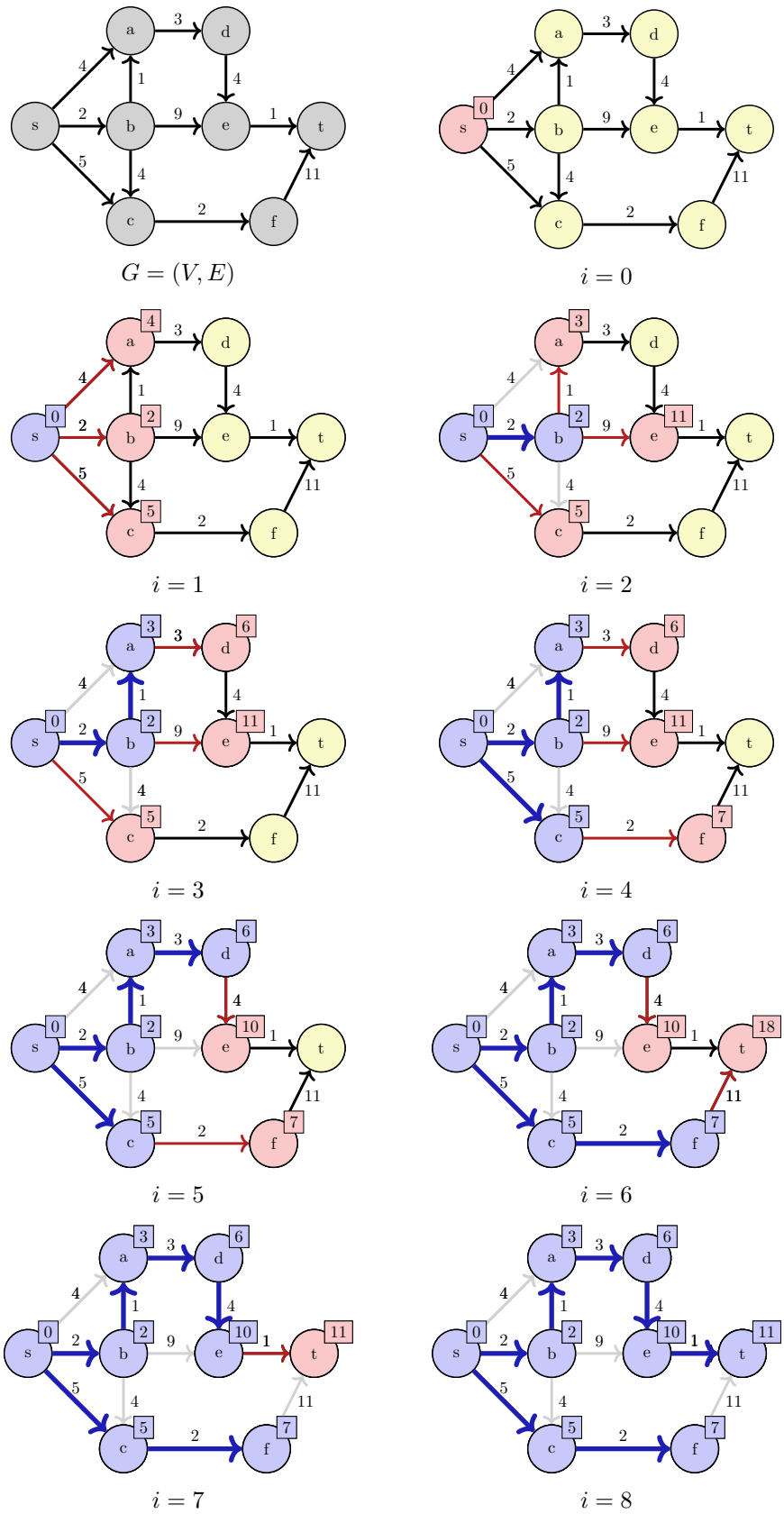


Figure 21.16: Dijkstra in action step for step, after every iteration.

Subsection 21.4

Improving Dijkstra

So far, we have seen two shortest paths algorithms: Breadth First Search (BFS) that works for unweighted graphs. Starting with s in a FIFO queue, it repeatedly removes the first node from the queue, visits it, and adds its yet-unseen out-neighbors to the queue, until the queue is empty, hence the whole graph (reachable from s) has been explored. Dijkstra's algorithm does the same, but uses a priority queue instead of a FIFO queue, with the path lengths as priority. It thus works for (positively) weighted graphs as well. Both algorithms have a monotonicity property: They explore the nodes in the order of increasing distance from the start node. Before a node with distance d is considered, *all* nodes with distance $d' < d$ have to be visited first. We can think of them exploring the graph in waves of increasing distance. See Figure 21.27.

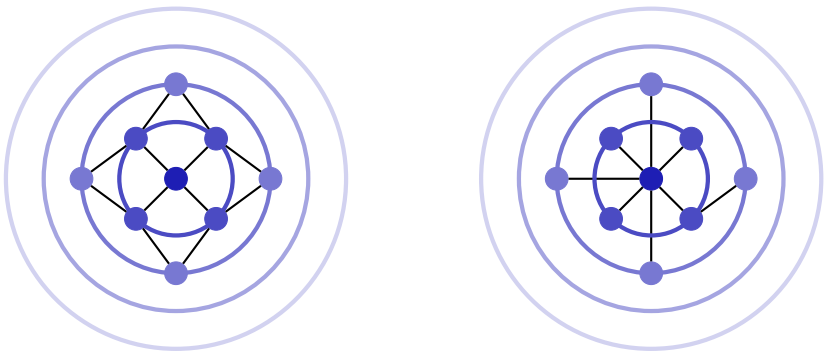


Figure 21.17: Both BFS and Dijkstra's algorithm explore the nodes in layers of increasing distance from the start node. While BFS only does it for unweighted graphs in terms of number of edges, Dijkstra achieves the same for (positively) weighted graphs by taking into account the edge weights. For weighted graphs, a node v connected to the start node s via a single edge might be explored later than a node w connected to s only via a path consisting of multiple edges. All that matters for the order of exploration is the weight of the shortest path (i.e., the sum of edge weights for the edges on the path).

This might make sense if the task is to explore the whole graph. However, if only a shortest path to a specific target node has to be found, this strategy seems to be unnecessarily complicated. If I want to find the shortest path from Zurich to Milano, why should I first find the shortest path to Stuttgart? While Stuttgart is closer to Zurich than Milano (is to Zurich), it lies in the opposite direction. Intuitively, unless there is a extremely cheap way to get from Stuttgart to Milano directly (not via Zurich), there is no point in exploring the area to the north of Zurich when looking for the shortest path to Milano, which lies in the south of Zurich. In practice, these unexpected shortcuts usually do not exist. It thus might make sense to favor places that we expect to be closer to our target. This is exactly what the **A* Algorithm (pronounced "A star")** does: It is a variant of Dijkstra's algorithm that adds a heuristic to prefer nodes that are expected to be closer to the target node.

21.4.1 A* Algorithm: A Heuristic for Dijkstra

For simplicity, we will restrict our attention to (undirected) grid graphs, as depicted in Figure 21.28. The algorithm is more general, but this simple case already captures the essence of the algorithm.

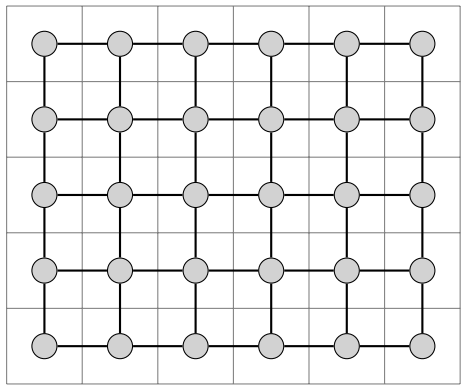


Figure 21.18: A grid graph of size 5×6. Instead of explicitly drawing nodes (circles) and edges (lines), we can think of the nodes being cells and two neighboring cells being connected via an edge. Both representations are depicted. In future discussions, we will often omit the explicit graph drawing.

Let us see what Dijkstra's algorithm does on this grid graph: It searches for all shortest paths to all nodes in all directions. See Figure 21.29.

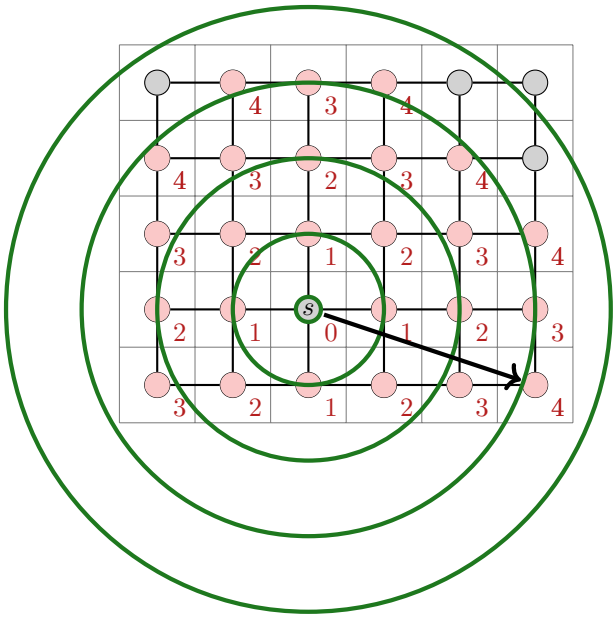


Figure 21.19: Dijkstra on a grid graph: Before the target node t (with distance 4) is visited, all nodes with distances $0, 1, 2$, and 3 from s are visited, no matter whether they lie in the opposite direction of t from s.

This is required if we do not have any further information about the distances. After all,

there might be a hidden shortcut somewhere. In particular, since we do not know the structure of the graph, it might be possible that on the direct "path" (the airline, so to speak) from s to t there is an obstacle, so that we have to take a detour in the wrong direction first. See Figure 21.30 for an example.

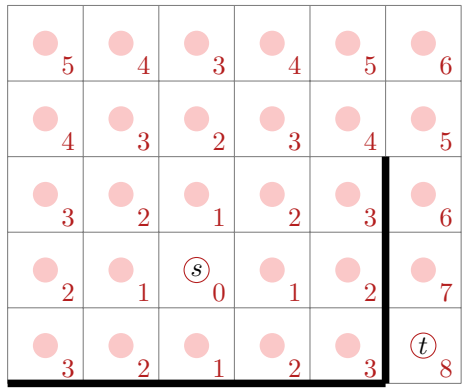
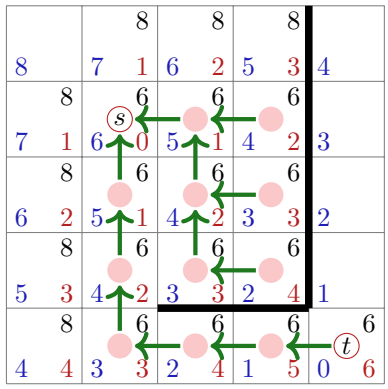


Figure 21.20: If there is an obstacle between s and t (indicated by the bold black line, which means that the cells separated by this black line are not connected via a direct edge), then the shortest path from s to t must first go in the opposite direction. This is why, for general graphs, Dijkstra is needed.

Nevertheless, it might be worth to explore nodes in the right direction (towards the target node) first. The idea of the **A* algorithm** is to use a heuristic—an estimate $\hat{h} \geq 0$ for the distance to t—to lead the search in the desired direction. More concretely, $\hat{h}(u)$ is an estimate for $\delta(u, t)$ (how far away this node u is from the target t). Intuitively, we want to favour nodes with small $\hat{h}$. Instead of working with the priorities d_s (the estimated distance from s, as Dijkstra does), we change the priorities to take $\hat{h}$ into account: We set the priority in the priority queue U to $\hat{f} := d_s + \hat{h}$, so we remove points closer to t sooner from U. The priority $\hat{f}(v)$ of a node v then corresponds to the estimated length of a path from s to t via this node v.

One common choice for $\hat{h}$ would be the airline distance, without taking into account obstacles. For the grid graph (with obstacles), we could use the Manhattan distance (the horizontal distance plus the vertical distance): $\hat{f} = \delta_x + \delta_y$. See Figure 21.31. This heuristic indeed leads to nodes in the opposite direction of the target not being visited.



$$\hat{f}(u) = d_s[u] + \hat{h}(u) \qquad (\hat{h} = \delta_x + \delta_y)$$

Figure 21.21: A* algorithm with Manhattan distance as heuristic $\hat{h}$. Upper bound d_s is given in red, heuristic in blue, the resulting priority (the sum) in black. At the time t is visited, the nodes with red dot have been discovered.

The pseudo-code of the A* algorithm can be found in Algorithm 21.3. Note that while $\hat{f}$ is used for the priority (so the order in which nodes are visited), they do not come into play for the computed distances d_s. We further note that we do not check whether a node has been removed from the priority queue U before adding it, so a node might be added to the priority queue repeatedly. This will become important later.

Input: Positively weighted Graph $G = (V, E, c)$, starting point $s \in V$, end point $t \in V$, estimate $\hat{h}$

Output: Existence and length of a shortest path from s to t

```
foreach u ∈ V do
    d_s[u] ← ∞; f̂[u] ← ∞; π[u] ← null
end
d_s[s] ← 0; f̂[s] ← ĥ(s); U ← {s}; S ← {}
while U ≠ ∅ do
    u ← ExtractMin_f̂(U); S ← S ∪ {u}
    if u = t then return success
    foreach v ∈ N⁺(u) with d_s[v] > d_s[u] + c((u,v)) do
        d_s[v] ← d_s[u] + c((u,v)); f̂[v] ← d_s[v] + ĥ(v); π[v] ← u
        U ← U ∪ {v}; S ← S \ {v}
    end
end
return failure
```

Algorithm 21.3: A*

Let us now introduce properties of the estimate $\hat{h}$ that determine whether (and if yes, how efficiently) the A* algorithm works. A heuristic can be an **optimistic estimate**. This means

$$\hat{h}(v) \leq \delta(v, t)$$

for all $v \in V$ (an under-estimate). This property is often also referred to as admissibility. We will see that it is crucial for correctness. Moreover, a heuristic can be **monotone** (sometimes

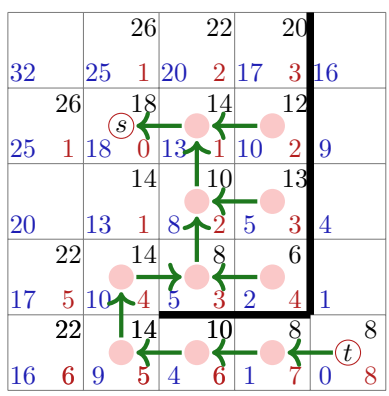
referred to as a *consistent* heuristic). This means that the inequality

$$\hat{h}(u) \leq c((u,v)) + \hat{h}(v)$$

holds for all $(u,v) \in E$. In other words, the estimated distance from u to t cannot be larger than the cost $c((u,v))$ from u to v plus the estimated distance from v to t. We will see that this property is crucial for the efficiency of the algorithm.[33]

First, suppose we have a heuristic that is not optimistic. So at least for certain nodes, our estimate is too pessimistic; it over-estimates the distance to t. Then the A* algorithm might terminate with the wrong result (even if the heuristic looks reasonable), as the example in Figure 21.32 shows. The problem is, intuitively, that a node v might have a small upper bound $d_s[v]$ from s, but—due to a high value of $\hat{h}(v)$—its priority $\hat{f}(v) = d_s[v] + \hat{h}(v)$ is too big for it to be considered. This leads to nodes being visited "too early". If we would keep running the algorithm after it has found t (for the first time), the error would be corrected eventually. However, this would defeat the point of the heuristic in the first place, as we would have to iterate until no changes happen anymore. This could be much easier achieved by Dijkstra without heuristic! The formal proof of this statement Lemma 21.4 can be found in Section 21.5.2 and more details in [HNR68].

Lemma 21.2. *If $\hat{h}$ is optimistic, then the A* algorithm terminates with the correct result: a shortest path from s to t.*



$\hat{f}(u) = d_s[u] + \hat{h}(u) \quad (\hat{h} = \delta_x^2 + \delta_y^2)$

Figure 21.22: The heuristic of the quadratic distance $\hat{h} = \delta_x^2 + \delta_y^2$ looks reasonable but it is not optimistic (it overestimates the distance to t for certain nodes). The algorithm terminates with a wrong result: a "shortest" path from s to t of length 8 instead of 6. The problem is that due to a too high estimate, certain crucial nodes (for instance, the one right below s) are removed from the priority queue way too late.

What if our heuristic is optimistic but not monotone, meaning that there is at least one edge $(u,v) \in E$ such that $\hat{h}(u) > c((u,v)) + \hat{h}(v)$. Then it could be that nodes have to be

[33] Note that for undirected graphs this can lead to a contradiction which is why monotonicity there is formulated as $|\hat{h}(u) - \hat{h}(v)| \leq c((u,v))$.

removed from U and re-added to U again and again. See Figure 21.33.

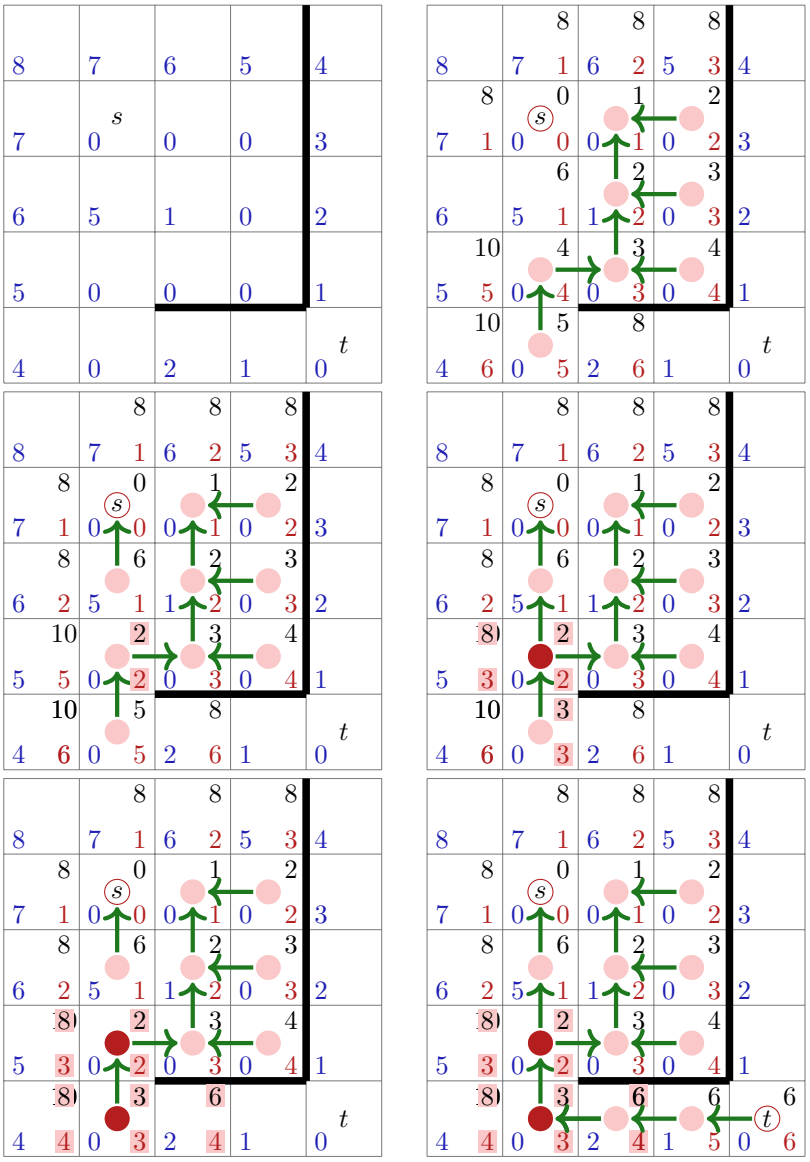


Figure 21.23: A non-monotone heuristic (e.g., the edge between the nodes one below s and two below s is non-monotone). Let us use u to denote the node below s and v to denote the node below u. Here, v is removed from U before u. When later v is removed from u, a better bound for v is found. The node v has to be added to the priority queue again. In the end, a shortest path is found, but two nodes have been added to the priority queue twice, which has a bad effect on the running time.

If $\hat{h}$ is monotone, then this repeated removal cannot happen. We observe that the A* algorithm is equivalent to Dijkstra's algorithm with edge weights $c'((u,v)) := c((u,v)) + \hat{h}(u) - \hat{h}(v)$. In particular, if $\hat{h} \equiv 0$, then A* algorithm is Dijkstra's algorithm (on the same graph).

In conclusion: If $\hat{h}$ is optimistic and monotone, then the A* algorithm is correct and efficient. In practice, when under-estimates are easy to compute (e.g., airline), this can lead to drastic improvement of Dijkstra's algorithm. Unfortunately, from a theory perspective, there are no results.

21.4.2 Proof of Lemma 21.4*

We first introduce a bit of notation. We use $f(v)$ to denote the length of a shortest path from s to t via v, so $f(v) := \delta(s,v) + \delta(v,t)$, remembering the property of optimal substructure (any subpath of a shortest path has to be a shortest path). See Figure 21.34.

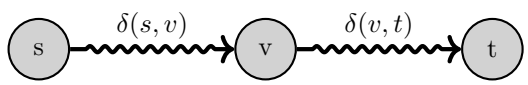


Figure 21.24: A shortest path from s to t via v has length $f(v) := \delta(s,v) + \delta(v,t)$.

Observation 21.4. *We have $f(s) = \delta(s,t) \geq \hat{h}(s)$ and $f(t) = \delta(t,t) = 0$. Moreover, for all nodes v on a shortest path from s to t except for t, we have $f(v) = f(s) = \delta(s,t)$.*

Before formally proving Lemma 21.4, let us observe that there is always a node $u \in U$ for which the shortest distance has already been computed, i.e., for which $d_s[u] = \delta(s,u)$.

Lemma 21.3. *Let v be an arbitrary node in U and P be an arbitrary shortest path from s to v. Then there is a node u on P that is still in U and satisfies (i) $d_s[u] = \delta(s,u)$ and (ii) $\hat{f}(u) \leq f(s) = \delta(s,t)$ if $\hat{h}$ is optimistic.*

Proof. Let $P = (v_0 = s, v_1, \ldots, v_k = v)$. In the beginning of the algorithm, $s = v_0 \in U$ and $d_s[s] = 0 = \delta(s,s)$, so then (i) is true trivially. Moreover, due to the optimistic estimate, we have

$$\hat{f}(s) = d_s[s] + \hat{h}(s) = \hat{h}(s) \leq \delta(s,t),$$

so (ii) holds as well.

Now let us consider a later stage in the algorithm. Let w be the last node on P that is in S. Note that it cannot be v, as $v \in U$. Let u be the successor of w on P. See Figure 21.35.

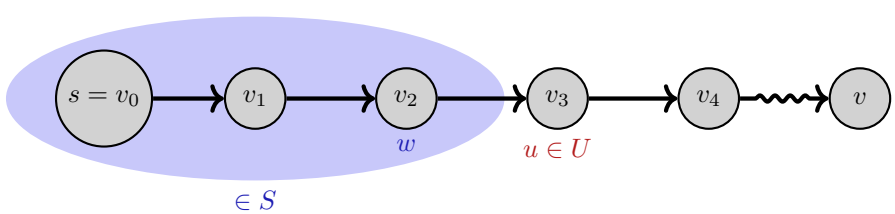


Figure 21.25: Let w be the last node in S and its successor v be the first node in U on a shortest path from s to v.

Due to the update rule of the algorithm, we have $d_s[u] \leq d_s[w] + c((w,u))$. Since $w \in S$, we have $d_s[w] = \delta(s,w)$. Moreover, as P is a shortest path, we have $\delta(s,u) = \delta(s,w) + c((w,u))$. Taken together, it thus follows that $d_s[u] \leq \delta(s,w) + c((w,u)) = \delta(s,u)$, and hence $d_s[u] = \delta(s,u)$, as the distance cannot be undershot. Thus, u indeed satisfies (i) and is in U, since it is a successor of a node in S but is not in S itself (as w was the latest such node).

To see (ii), observe that $\hat{f}(u) = d_s[u] + \hat{h}(u) \overset{(i)}{=} \delta(s,u) + \hat{h}(u) \leq \delta(s,u) + \delta(u,t) = \delta(s,t)$, where the second last step follows from the estimate being optimistic and the last step is due to P being an optimal path. □

We are now ready to prove Lemma 21.4.

Proof of Lemma 21.4. First, we need to argue that the algorithm terminates. Since every node can be added to U only a finite number of times, as there are only a finite number of paths from s to this node. Thus, eventually the algorithm will remove all nodes from U; the algorithm terminates. Moreover, if t is reachable from s, eventually t will be added before U is empty. Hence, the only way for our algorithm to be incorrect is for it to return an incorrect shortest path. More concretely, since the algorithm cannot undershoot the distance, it would need to terminate with a longer path, so with $d_s[t] > \delta(s,t)$.

Right before the algorithm terminates (with the wrong result $d_s[t] > \delta(s,t)$), there is a node $u \in U$ on a shortest path from s to t with $\hat{f}(u) \leq \delta(s,t)$ by Lemma 21.5 (ii). Moreover, we have $\hat{f}(t) = d_s[t] + \hat{h}(t) = d_s[t]$, so it follows $\hat{f}(t) > \hat{f}(u)$. However, since t has been removed from U in the last step (and not u), this leads to a contradiction, and hence concludes the proof. □

References

[BFP+73] Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.

[CLRS09] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, Third Edition*. The MIT Press, 3rd edition, 2009.

[HNR68] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

[Hoa74] C. A. R. Hoare. Monitors: an operating system structuring concept. *Commun. ACM*, 17(10):549–557, oct 1974.

[Knu98] D.E. Knuth. *The Art of Computer Programming: Sorting and Searching, Volume 3*. Pearson Education, 1998.

[OW17] T. Ottmann and P. Widmayer. *Algorithmen und Datenstrukturen*. Springer Berlin Heidelberg, 2017.

[Str13] B. Stroustrup. *The C++ Programming Language*. Always learning. Addison-Wesley, 2013.

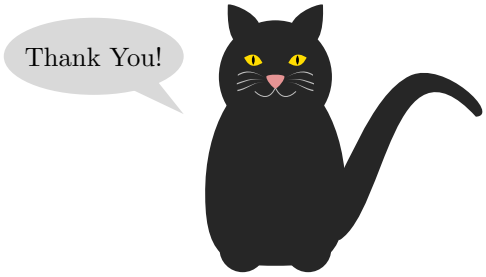
[Str22] B. Stroustrup. *A Tour of C++*. C++ In-Depth Series. Pearson Education, 2022.

Section 2

Acknowledgements

We received feedback and bug reports from (in alphabetical order)

- Lukas Achermann
- Isaac van Bakel
- Patrik Brajkovic
- Yutong Chen
- Sabine Dankelschijn
- Carmen Engele
- Thomas Ewald
- Adel Gavranović
- Johannes Hostert
- Janosch Jörg
- Philipp Junghans
- Arghavan Kassraie
- Alexander Kent
- Gian Laager
- Ritwik Majumdar
- Gabriele Olendraite
- Alexander Pietak
- Frano Rajič
- Ece Sar
- Leon Thomm
- Felix Witschi
- Tim Wyss
- Marcel Zbinden



Want to be on this page? Report bugs or give feedback via e-mail to dsaln@lists.inf.ethz.ch.